

LIBRARY MANAGEMENT SYSTEM

NAME

Mail id

ABSTRACT

The Library Management System is a database-centric solution developed exclusively in Oracle PL/SQL to automate core operations of academic and public libraries. Traditional library methods relying on manual entries suffer from severe data inconsistencies, slow tracking of issued books, and inventory errors. This system removes human errors by handling operations natively inside the database engine. The design relies on interconnected tables: Books tracking inventory, Members storing subscriber data, and Issue_Logs recording operational history. By embedding business rules in stored procedures, the system validates availability before issuing items. A database trigger automatically updates physical inventory numbers instantly when a transaction occurs, maintaining strict reference integrity. The system uses PL/SQL exception blocks to catch operational errors, such as non-existent members or stock depletion, preventing runtime database crashes.

By centralizing operational rules inside the database layer, this project minimizes data round-trips and guarantees data safety. Data modification runs entirely inside safe database transactions. This ensures critical updates execute completely or roll back on errors, keeping data uniform. The architecture bypasses external application layers, using standard input/output lines to communicate directly with terminal clients. This terminal focus demonstrates how backend systems run production workflows using only relational triggers and stored procedures. This project establishes a reliable template for beginners learning enterprise data persistence, constraints, and backend transaction flows.

CODING

DATABASE SCHEMA (DDL)

```
CREATE TABLE books (  
    book_id NUMBER PRIMARY KEY,  
    title VARCHAR2(100),  
    author VARCHAR2(100),  
    total_copies NUMBER,  
    available_copies NUMBER  
);  
  
CREATE TABLE members (  
    member_id NUMBER PRIMARY KEY,  
    member_name VARCHAR2(100),  
    join_date DATE DEFAULT SYSDATE  
);  
  
CREATE TABLE issue_logs (  
    issue_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    book_id NUMBER REFERENCES books(book_id),  
    member_id NUMBER REFERENCES members(member_id),  
    issue_date DATE DEFAULT SYSDATE,  
    return_date DATE  
);  
  
-- Seed Data  
INSERT INTO books VALUES (101, 'Oracle PL/SQL Programming', 'Steven  
Feuerstein', 5, 5);  
INSERT INTO books VALUES (102, 'Database System Concepts',  
'Silberschatz', 3, 3);  
INSERT INTO members VALUES (501, 'Alice Smith', SYSDATE);  
INSERT INTO members VALUES (502, 'Bob Jones', SYSDATE);  
COMMIT;
```

CORE PROCEDURE (ISSUE BOOK)

```
CREATE OR REPLACE PROCEDURE issue_book (  
    p_book_id IN NUMBER,  
    p_member_id IN NUMBER  
) IS  
    v_available NUMBER;  
BEGIN  
    SELECT available_copies INTO v_available FROM books WHERE  
book_id = p_book_id;  
  
    IF v_available > 0 THEN  
        INSERT INTO issue_logs (book_id, member_id, issue_date)  
VALUES (p_book_id, p_member_id, SYSDATE);  
        COMMIT;  
        DBMS_OUTPUT.PUT_LINE('Book issued successfully to member: '  
|| p_member_id);  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('Error: Book is currently out of  
stock.');
```

```
    END IF;  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('Error: Book ID or Member ID not  
found.');
```

```
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Unexpected Error: ' || SQLERRM);  
END;  
/
```

DATABASE TRIGGER (AUTO-INVENTORY DECREASE)

```
CREATE OR REPLACE TRIGGER trg_after_issue
AFTER INSERT ON issue_logs
FOR EACH ROW
BEGIN
    UPDATE books
    SET available_copies = available_copies - 1
    WHERE book_id = :NEW.book_id;
END;
/
```

CONCLUSION

The Library Management System proves that standalone PL/SQL architecture handles operational inventory management effectively. Moving critical constraints out of application layers and into the relational database engine enhances security and system speed. Using database triggers guarantees real-time stock balance updates without depending on external software logic. This design successfully prevents common tracking errors like overselling stock or double-booking resources. The project showcases how stored procedures reduce corporate network strain by processing heavy operations on the database server. Ultimately, this backend-first design establishes an isolated, consistent database foundation. This core is stable enough to safely connect with web or mobile frontends in future development phases.

FUTURE ENHANCEMENT

Future iterations can expand this architecture by building an automatic fine calculation schedule using Oracle JOB or DBMS_SCHEDULER. This will analyze overdue logs daily without manual triggers. Adding custom object types can track multi-format inventory items, such as digital e-books, multimedia CDs, and research journals. Integrating Oracle APEX will quickly transition the backend terminal into a modern web portal. Furthermore, adding automated email notification triggers via the UTL_SMTP utility will alert members immediately about due dates. Finally, implementing fine collection processing using secure transaction APIs will turn this project into a comprehensive, enterprise-ready digital platform.